

Código de test de las IRQ en el P1

formato aritméticas:

ADD RD, RS, RT

formato lw, sw, beq:

LW RT, INM(RS)

SW RT, INM(RS)

BEQ RS, RT

Contenido Memoria Datos: [256, 1, 8, Nint (0 al empezar), 12, 0x00000AB0, 0, 0x0BAD0C0D, 0, 0, 0...]

Pseudo-código

```
const int sp_init = 256;
const int one = 1;
const int eight = 8;
volatile static int Nint = 0;

void main(){
    register int R1;
    Print(R1); // Escribe un entero en el registro de salida
    while(1) {R1=2*R1;Print(R1);} // Escribe un entero en el registro de salida

    void RTI (void) __irq { //__irq indica que es una excepción y que hay que
        Nint++; // que generar el prólogo y epílogo adecuado
        Print(Nint);} // Escribe un entero en el registro de salida

    void Abort (void) __abort{//__abort indica que es una excepción y que hay
        while(1){};} // que generar el prólogo y epílogo adecuado

    void UNDEF (void) __undef {//__undef indica que es una excepción y que hay
        while(1){};} // que generar el prólogo y epílogo adecuado
```

Valores finales

r1=1,2,4,8,16....

Mem(C)= 0,1,2,3,4.....

Reset	@0x0	10210003	beq R1, R1, INI;	Se salta siempre a la @16 donde empieza el programa (@0x8)
IRQ	@0x4	1021003E	beq R1, R1, RTI;	Se salta siempre a la @64*4
DAbort	@0x8	1021005D	beq R1, R1, RT Abort;	Se salta siempre a la @96*4
UNDEF	@0xC	1021006C	beq R1, R1, RT_UNDEF;	Se salta siempre a la @112*4
INI:	@0x10	081F0000	LW R31, 0(R0)	R31=Mem(0) = 256; R31 es el puntero de pila (SP)
		08010004	LW R1, 4(R0)	R1=Mem(4) = 1;
		0CC17004	sw r1, 7004(r6)	IO output <= 1; ld-uso
Main		04210800	ADD R1, R1, R1	R1=2*R1
		0CC17004	sw r1, 7004(r6)	IO output <= R1; 2,4,8,16...
		1021FFFD	beq R1, R1, main	Bucle infinito. Solo se sale si hay una IRQ
RTI:	@0x100	0FE10000	SW R1, 0(R31)	Guardamos el contenido de R1 en pila
		0FE20004	SW R2, 4(R31)	Guardamos el contenido de R2 en pila
		08010008	LW R1, 8(R0)	R1=Mem(8) = 8;
		07E1F800	ADD r31, R31, R1	R31=R31 +8; Incrementamos el SP
		0802000C	LW R2, C(R0)	R2=Mem(C) En esta posición de memoria contabilizamos el número de int (Nint)
		08010004	LW R1, 4(R0)	R1=Mem(4) = 1;
		04221000	ADD R2, R1, R2	R2= Nint++
		0CC27004	sw r2, 7004(r6)	IO output <= R2
		0C02000C	SW R2, C(R0)	Mem(C) = Nint++
		0CC17008	sw r1, 7008(r6)	INT ACK <=1; (dura un ciclo)
		08010008	LW R1, 8(R0)	R1=Mem(8) = 8;
		07E1F801	SUB r31, R31, R1	SP=SP-8
		0BE10000	LW R1, 0(R31)	Restauramos el contenido de R1 de pila
		0BE20004	LW R2, 4(R31)	Restauramos el contenido de R2 de pila
		20000000	rte	Se vuelve a la instrucción que se interrumpió
		08C17FFF	LW R1, 24000(R6)	Abort por fuera de rango. Como esta instrucción se invalida no debería generar excepción
RT_Abort	@0x180	08C10014	LW R1, 20(R6)	R1=Mem(20) =0x00000AB0;
		0CC17004	sw r1, 7004(r6)	IO output <=0x00000AB0;
	bucleA	1000FFFF	beq R0, R0, RT Abort	Bucle infinito
RT_UNDEF	@0x1C0	08C1001C	LW R1, 24(R6)	R1=Mem(28) = 0x0BAD0COD;
		0CC17004	sw r1, 7004(r6)	IO output <=0x00000AB0;
	bucleU	1000FFFF	beq R0, R0, RT_UNDEF	Bucle infinito